

CSE 402

University of Asia Pacific

Course Name : Operating System Lab

Lab Report 4

Topic : **Disk Scheduling (FCFS and SCAN)**

Presented by,

Fahad Hasan Mahi
Reg : 23101186
Section : D1

Presented to,

Atia Rahman Orthi
Lecturer, UAP

Problem Statement :

Consider a disk drive with **200 cylinders (0–199)**. We are given a queue of disk I/O requests and an initial disk head position. The goal is to simulate two disk scheduling algorithms to determine the total distance the disk arm moves (Total Head Movement) and the average distance per request (Average Seek Length).

Dataset 1 (FCFS):

- Initial Head Position: 53
- Request Queue: **[98, 183, 37, 122, 14, 124, 65, 67]**
- Algorithm: First-Come, First-Served (FCFS) - Service requests strictly in the order they arrive.

Dataset 2 (SCAN):

- Initial Head Position: 50
- Request Queue: **[98, 183, 37, 122, 14, 124, 65, 67]**
- Disk Size: 200
- Direction: Left (towards 0)
- Algorithm: SCAN (Elevator Algorithm) - Service requests in the current direction until the boundary is reached, then reverse direction.

Source Code :

FCFS & SCAN Disk Scheduling-

```
def fcfs_disk_scheduling(request_queue, initial_head):
 total_head_movement = 0
 current_head = initial_head

 print("FCFS Disk Scheduling")
 print(f"Initial Head Position: {initial_head}")
 print(f"Request Queue: {request_queue}\n")

 print("Step\tCurrent Track\tNext Track\tHead Movement")
 print("-" * 55)

 for i in range(len(request_queue)):
 next_track = request_queue[i]
 distance = abs(next_track - current_head)
 total_head_movement += distance

 print(f"{i+1}\t{current_head}\t\t{next_track}\t\t{distance}")
 current_head = next_track
```

```
print("-" * 55)
print(f"Total Head Movement: {total_head_movement} tracks")
avg_seek = total_head_movement / len(request_queue)
print(f"Average Seek Length: {round(avg_seek, 2)} tracks\n")


def scan_disk_scheduling(request_queue, initial_head, disk_size, direction='left'):
total_head_movement = 0
current_head = initial_head
seek_sequence = []

request_queue.sort()
left = [r for r in request_queue if r < initial_head]
right = [r for r in request_queue if r >= initial_head]

if direction == 'left':
seek_sequence.extend(reversed(left))
if 0 not in seek_sequence:
seek_sequence.append(0)
seek_sequence.extend(right)
else:
seek_sequence.extend(right)
if disk_size not in seek_sequence:
seek_sequence.append(disk_size)
seek_sequence.extend(reversed(left))

print(" SCAN Disk Scheduling (Direction: left) ")
print(f"Initial Head: {initial_head}, Disk Size: {disk_size}")
print(f"Seek Sequence: {seek_sequence}\n")

print("Step\tCurrent Track\tNext Track\tHead Movement")
print("-" * 55)

prev_track = initial_head
for i, track in enumerate(seek_sequence):
distance = abs(track - prev_track)
total_head_movement += distance
print(f"{i+1}\t{prev_track}\t\t{track}\t\t{distance}")
prev_track = track

print("-" * 55)
print(f"Total Head Movement: {total_head_movement} tracks")
print(f"Average Seek Length: {round(total_head_movement / len(request_queue), 2)} tracks")



request_queue = [98, 183, 37, 122, 14, 124, 65, 67]
```

fcfs_disk_scheduling(request_queue, initial_head=53)
scan_disk_scheduling(request_queue, initial_head=50, disk_size=200, direction='left')

Output Screenshot :

```
FCFS Disk Scheduling
Initial Head Position: 53
Request Queue: [98, 183, 37, 122, 14, 124, 65, 67]

Step    Current Track   Next Track      Head Movement
-------------------------------------------------------
1       53              98              45
2       98              183             85
3       183             37              146
4       37              122             85
5       122             14              108
6       14              124             110
7       124             65              59
8       65              67              2
-------------------------------------------------------
Total Head Movement: 640 tracks
Average Seek Length: 80.0 tracks
```

Fig 1: FCFS disk scheduling

```
 SCAN Disk Scheduling (Direction: left)
Initial Head: 50, Disk Size: 200
Seek Sequence: [37, 14, 0, 65, 67, 98, 122, 124, 183]

Step    Current Track   Next Track      Head Movement
-------------------------------------------------------
1       50              37              13
2       37              14              23
3       14              0               14
4       0               65              65
5       65              67              2
6       67              98              31
7       98              122             24
8       122             124             2
9       124             183             59
-------------------------------------------------------
Total Head Movement: 233 tracks
Average Seek Length: 29.12 tracks
```

Fig 2: SCAN disk scheduling

Comparison: FCFS vs SCAN:

Numeric :

Metric	FCFS	SCAN
Total Head Movement	640 tracks	233 tracks
Average Seek length	80.0 tracks	29.12 tracks
Seek Sequence Variance	Extremely High (swings from 183 to 37)	Low (moves smoothly in one direction)

Feature Comparison :

Aspect	FCFS	SCAN
Selection	Order of arrival	Direction of head movement + proximity
Preemption Trigger	Shorter Job arrives	Time Quantum Expires
Starvation	Not Possible	Possible
Head Movement	Unpredictable, often excessive	Bounded, predictable
Fairness	Fair (prefers short jobs)	Slightly Biased toward middle cylinders
Use Case	Light disk loads	Heavy disk loads

Discussion :

The simulation results highlight a massive efficiency gap between the two algorithms. FCFS produced **640 tracks** of total head movement, largely because it blindly followed the arrival order, causing the disk arm to swing wildly across the platter (e.g., jumping from cylinder 183 all the way down to 37, a 146-track swing). SCAN reduced this movement to just **233 tracks** (a 63.6% improvement) by organizing the requests into a smooth, elevator-like sweep towards cylinder 0 before reversing.

However, this efficiency comes with a trade-off in fairness and boundary handling. In FCFS, every request is guaranteed to be serviced in the order it arrived, making it completely starvation-free. In SCAN, the algorithm forces the disk head to travel all the way to the physical boundary (cylinder 0 in this case) even if there are no requests there. This "boundary bounce" adds a small overhead but ensures that requests at the

extreme ends of the disk are eventually serviced, though they may experience longer wait times compared to requests in the middle.

Conclusion :

This lab demonstrated that while FCFS is the simplest and fairest disk scheduling algorithm, it is highly inefficient for heavy workloads due to excessive and unpredictable head movement. SCAN significantly optimizes performance by reducing total head movement and average seek time, making it far superior for systems with high I/O demand. However, SCAN introduces a slight bias against requests at the extreme ends of the disk and requires the head to travel to physical boundaries even when unnecessary. Therefore, FCFS remains suitable for light, low-priority background tasks where fairness is paramount, while SCAN is the preferred choice for general-purpose operating systems where overall throughput and seek efficiency are critical.

Github repository : https://github.com/fhmahi957/CSE_402_Assignment_materials.git